

5. Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

OUTPUT :

```
#include<stdio.h>
#include<unistd.h>
#include<time.h>
int main() {
    int fd[2]; //declaring the pipe
    pipe(fd); //creating the pipe
    int pid, data; //declaring the variables for data and pid
    pid=fork(); //creating the process
    if(pid==0) //checking for child
    {
        printf("Consumer \n"); //child is not writing its reading so close the write end of pipe anything so we can close
        close(fd[1]); //child is not writing its reading so close the write
        for(int i=0; i<5; i++) //5 items in consumer so it produces the consumer
        {
            read(fd[0], &data, sizeof(data));
            printf("Consumed %d\n", data);
        }
        close(fd[0]);
    }
    else
    {
        printf("Producer\n");
        close(fd[0]);
        for(int i=1; i<=5; i++)
        {
            data=i*10;
            write(fd[1], &data, sizeof(data));
            printf("Produced %d\n", data);
        }
        close(fd[1]);
    }
}
```

```
hemaswi@Hemaswi:~$ nano practical5A.c
hemaswi@Hemaswi:~$ gcc practical5A.c -o practical5A
hemaswi@Hemaswi:~$ ./practical5A
Producer
Produced 10
Produced 20
Produced 30
Produced 40
Produced 50
Consumer
Consumed 10
Consumed 20
Consumed 30
Consumed 40
Consumed 50
hemaswi@Hemaswi:~$
```

5(B) :

OUTPUT :

```
#include<stdio.h>
#include<unistd.h>
#include<time.h>
int main() {
int fd[2]; //declaring the pipe
pipe(fd); //creating the pipe
int pid,data; //declaring the variables for data and pid
int start,end;
double timetaken;
pid=fork(); //creating the process
start=clock();
if(pid==0) //checking for child
{
printf("Consumer \n"); //child is not writing its reading so close the write e>
close(fd[1]); //child is not writing its reading so close the write
for(int i=0;i<5;i++) //5 items in consumer so it produces the consumer
{
read(fd[0],&data,sizeof(data));
printf("Consumed %d\n",data);
}
close(fd[0]);
}
else
{
printf("Producer \n");
close(fd[0]);
for(int i=1;i<=5;i++)
{
data = i*10;
write(fd[1],&data,sizeof(data));
printf("Produced %d\n",data);
}
close(fd[1]);
end=clock();
timetaken=(double)(end-start)/CLOCKS_PER_SEC;
printf("Time taken is %f",timetaken);
printf("Communication Time is %f",timetaken);
}
}
```

```
hemaswi@Hemaswi:~$ nano practical5B.c
hemaswi@Hemaswi:~$ gcc practical5B.c -o practical5B
hemaswi@Hemaswi:~$ ./practical5B
Producer
Consumer
Produced 10
Produced 20
Consumed 10
Produced 30
Consumed 20
Produced 40
Consumed 30
Produced 50
Consumed 40
Consumed 50
Time taken is 0.000131Communication Time is 0.000131hemaswi@Hemaswi:~$
```

5(C):

OUTPUT:

```
#include <stdio.h>
#include <unistd.h>
#include <time.h>
int main()
{
    int fd[2]; // declaring the pipe
    pipe(fd); // creating the pipe
    int pid, data;
    int start, end;
    double timetaken, efficiency;
    pid = fork(); // creating process
    start = clock();
    if(pid == 0) // child
    {
        printf("Consumer\n");
        close(fd[1]); // close write end
        for(int i = 0; i < 5; i++)
        {
            read(fd[0], &data, sizeof(data));
            printf("Consumed %d\n", data);
        }
        close(fd[0]);
    }
    else
    {
        printf("Producer\n");
        close(fd[0]); // close read end
        for(int i = 1; i <= 5; i++)
        {
            data = i * 10;
            write(fd[1], &data, sizeof(data));
            printf("Produced %d\n", data);
        }
        close(fd[1]);
        end = clock();
        timetaken = (double)(end - start) / CLOCKS_PER_SEC;
        efficiency = 5 / timetaken;
        printf("Communication Time is %f sec\n", timetaken);
        printf("Communication Efficiency = %f items/sec\n", efficiency);
    }
}
```

```
hemaswi@Hemaswi:~$ nano practical5C.c
hemaswi@Hemaswi:~$ gcc practical5C.c -o practical5C
hemaswi@Hemaswi:~$ ./practical5C
Producer
Produced 10
Produced 20
Produced 30
Produced 40
Produced 50
Communication Time is 0.000099 sec
Communication Efficiency = 50505.050505 items/sec
Consumer
Consumed 10
Consumed 20
Consumed 30
Consumed 40
Consumed 50
hemaswi@Hemaswi:~$ |
```